🏆 MODULE 6 — Intelligent Country Ranking System

ML-Based Comparative Analysis of Global Education Systems

> **Goal:** Generate an AI-driven country ranking using ML predictions + weighted feature importance — NOT manual scoring.

How the ranking works:

- Train Random Forest on student data (R^2 = 0.90)
- Extract feature importance weights
- Predict performance score for each country profile
- Compute a weighted composite score using feature importance
- Blend both scores → Final Ranking Score (0–100 scale)
- **Dynamic mode:** change any educational variable → ranking updates automatically

Steps:

1. Install & import libraries
2. Upload dataset
3. Train ML model
4. Build base ranking
5. Visualize ranking (4 charts)
6. Dynamic ranking simulator (India reform demo)
7. Download results

⚙️ Step 1 — Install & Import Libraries

```
!pip install -q scikit-learn pandas numpy matplotlib seaborn
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.gridspec as gridspec
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error
import warnings
warnings.filterwarnings('ignore')
```

```
np.random.seed(42)
print('✅ Libraries imported!')
```

```
✅ Libraries imported!
```

📂 Step 2 — Upload Dataset

```
from google.colab import files
import io

print('📁 Upload your student_performance_global.csv file...')
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(io.BytesIO(uploaded[filename]))

print(f"\n✅ '{filename}' loaded!")
print(f'   Rows: {df.shape[0]} | Columns: {df.shape[1]}')
print(f'   Countries: {sorted(df["country"].unique().tolist())}')
df.head(3)
```

```
📁 Upload your student_performance_global.csv file...
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving student_performance_global.csv to student_performance_global.csv

✅ 'student_performance_global.csv' loaded!
   Rows: 650 | Columns: 41
   Countries: ['Canada', 'China', 'Finland', 'Germany', 'India', 'Japan', 'Si
```

	student_id	country	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjc
0	1	India	F	15	R	LE3	T	3.90	2.80	oth
1	2	India	M	19	U	LE3	T	4.09	4.65	teach
2	3	India	M	18	U	LE3	T	3.42	3.77	oth

3 rows × 41 columns

🔧 Step 3 — Configure Features & Train ML Model

```
# ── Feature configuration ──────────────────────────────────────────
FEATURES = [
    'math_score_index',
    'reading_score_index',
    'science_score_index',
    'school_quality_index',
    'govt_edu_spend_index',
    'teacher_ratio_index',
    'student_wellbeing_index',
    'studytime',
```

```
    'failures',
    'absences',
]
FEATURE_LABELS = {
    'math_score_index':        'Math Score',
    'reading_score_index':     'Reading Score',
    'science_score_index':     'Science Score',
    'school_quality_index':    'School Quality',
    'govt_edu_spend_index':    'Govt. Spend',
    'teacher_ratio_index':     'Teacher Quality',
    'student_wellbeing_index': 'Well-being',
    'studytime':               'Study Time',
    'failures':                'Past Failures',
    'absences':                'Absences',
}
# Features that HURT performance (higher = worse)
NEGATIVE_FEATURES = ['failures', 'absences']
TARGET = 'G3'

# Validate columns
missing = [f for f in FEATURES if f not in df.columns]
if missing:
    print(f'⚠️  Skipping missing columns: {missing}')
    FEATURES = [f for f in FEATURES if f in df.columns]

# ── Train Random Forest ──────────────────────────────────────────────
model_df = df[FEATURES + [TARGET, 'country']].dropna()
X = model_df[FEATURES]
y = model_df[TARGET]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

rf = RandomForestRegressor(n_estimators=200, random_state=42, max_depth=8)
rf.fit(X_train, y_train)

r2   = r2_score(y_test, rf.predict(X_test))
rmse = np.sqrt(mean_squared_error(y_test, rf.predict(X_test)))

# Feature importance weights (normalized to sum = 1)
fi_norm = pd.Series(
    rf.feature_importances_ / rf.feature_importances_.sum(),
    index=FEATURES
)

# ── Compute BASE reference scale ─────────────────────────────────────
# IMPORTANT: We fix min/max from the BASE dataset.
# This ensures dynamic scenarios produce scores on the SAME scale,
# so changes in India's features actually shift its score.
base_profiles = df.groupby('country')[FEATURES].mean()
base_pred_raw = rf.predict(scaler.transform(base_profiles))
base_pred_s   = pd.Series(base_pred_raw, index=base_profiles.index)
```

```
base_comp = pd.Series(0.0, index=base_profiles.index)
for feat in FEATURES:
    if feat in NEGATIVE_FEATURES:
        base_comp -= fi_norm[feat] * base_profiles[feat]
    else:
        base_comp += fi_norm[feat] * base_profiles[feat]

PRED_MIN, PRED_MAX = base_pred_s.min(), base_pred_s.max()
COMP_MIN, COMP_MAX = base_comp.min(), base_comp.max()

print(f'✅ Random Forest trained — R² = {r2:.4f} | RMSE = {rmse:.4f}')
print(f'   Training samples: {len(X_train)} | Test samples: {len(X_test)}')
print('\n📊 Feature Importance Weights (used for composite score):')
fi_table = pd.DataFrame({
    'Feature': [FEATURE_LABELS[f] for f in fi_norm.index],
    'Weight':  fi_norm.round(4).values,
    'Impact':  ['NEGATIVE' if f in NEGATIVE_FEATURES else 'POSITIVE' for f i
}).sort_values('Weight', ascending=False).reset_index(drop=True)
fi_table.index += 1
display(fi_table.style.background_gradient(subset=['Weight'], cmap='YlGn'))
```

```
✅ Random Forest trained — R² = 0.9046 | RMSE = 0.4359
   Training samples: 520 | Test samples: 130

📊 Feature Importance Weights (used for composite score):
```

	Feature	Weight	Impact
1	Science Score	0.407300	POSITIVE
2	Math Score	0.333200	POSITIVE
3	Past Failures	0.082700	NEGATIVE
4	Reading Score	0.039800	POSITIVE
5	School Quality	0.031900	POSITIVE
6	Govt. Spend	0.027400	POSITIVE
7	Well-being	0.025700	POSITIVE
8	Teacher Quality	0.023000	POSITIVE
9	Absences	0.017800	NEGATIVE
10	Study Time	0.011100	POSITIVE

🏆 Step 4 — Build Base Country Ranking

Ranking Formula:

```
Final Score = 60% x ML Predicted Score  +  40% x Weighted Composite Score
```

Both scores are normalized to a 60–100 scale using fixed base-dataset min/max.

```
def compute_ranking(df_input):
    """
    Compute country rankings using fixed base scale.
    df_input: full student-level dataframe (possibly modified for scenarios)
    Returns: DataFrame sorted by rank.
    """
    profiles = df_input.groupby('country')[FEATURES].mean()

    # ML predicted score
    pred_raw = rf.predict(scaler.transform(profiles))
    pred_s   = pd.Series(pred_raw, index=profiles.index)

    # Weighted composite score
    comp = pd.Series(0.0, index=profiles.index)
    for feat in FEATURES:
        if feat in NEGATIVE_FEATURES:
            comp -= fi_norm[feat] * profiles[feat]
        else:
            comp += fi_norm[feat] * profiles[feat]

    # Normalize using FIXED base scale (60-100 range)
    pred_norm = ((pred_s - PRED_MIN) / (PRED_MAX - PRED_MIN) * 40 + 60).roun
    comp_norm = ((comp  - COMP_MIN) / (COMP_MAX - COMP_MIN) * 40 + 60).round

    final = (0.6 * pred_norm + 0.4 * comp_norm).round(2)

    result = pd.DataFrame({
        'ML Predicted Score':    pred_norm,
        'Composite Score':       comp_norm,
        'Final Score':           final,
        'Actual G3':             df_input.groupby('country')[TARGET].mean().
    })
    result['Rank'] = result['Final Score'].rank(ascending=False).astype(int)
    return result.sort_values('Rank')

# Compute base ranking
base_ranking = compute_ranking(df)

print('🏆 AI-Generated Country Education Ranking\n')
print(f'{"Rank":<6} {"Country":<15} {"Final Score":<14} {"ML Score":<12} {"C
print('-' * 72)
medals = {1:'🥇', 2:'🥈', 3:'🥉'}
for country, row in base_ranking.iterrows():
```

```
    medal = medals.get(row['Rank'], '    ')
    flag  = ' 🔴' if country == 'India' else ''
    print(f"{medal} {int(row['Rank']):<4} {country:<15} {row['Final Score']:
          f"{row['ML Predicted Score']:<12} {row['Composite Score']:<12} {ro

print('\n💡 Key observations:')
india_score = base_ranking.loc['India', 'Final Score']
top_score   = base_ranking['Final Score'].max()
gap         = top_score - india_score
print(f'   India score: {india_score} | Top score: {top_score} | Gap: {gap:.
print(f'   India ranks last — confirming the need for systemic reform.')
```

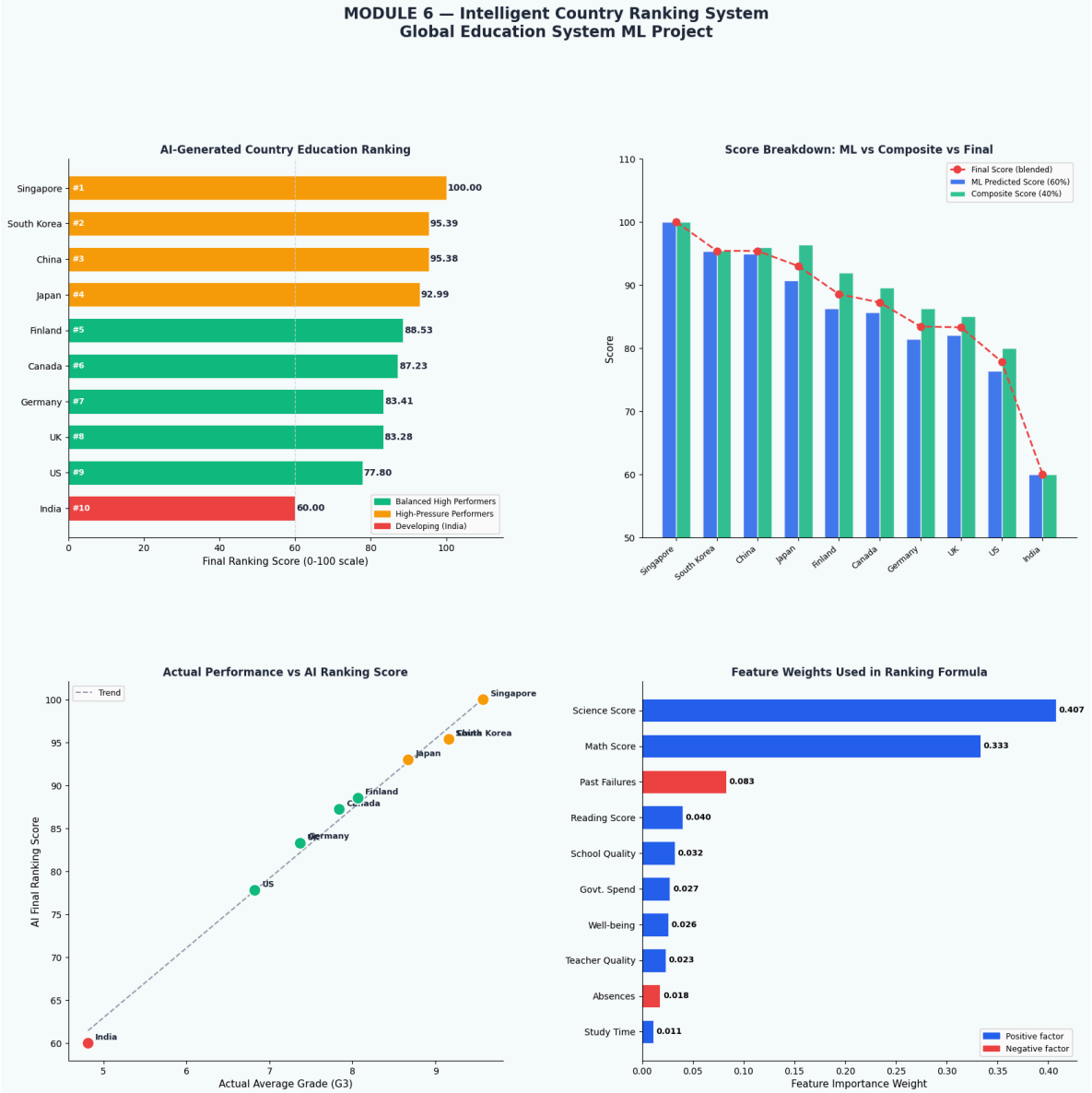
```
🏆 AI-Generated Country Education Ranking

Rank   Country         Final Score    ML Score     Composite    Actual G3
----------------------------------------------------------------------
🥇 1    Singapore       100.0          100.0        100.0        9.575
🥈 2    South Korea     95.39          95.37        95.42        9.157
🥉 3    China           95.38          94.96        96.0         9.163
   4    Japan           92.99          90.73        96.37        8.672
   5    Finland         88.53          86.23        91.98        8.068
   6    Canada          87.23          85.7         89.53        7.841
   7    Germany         83.41          81.48        86.31        7.379
   8    UK              83.28          82.1         85.04        7.371
   9    US              77.8           76.35        79.98        6.823
   10   India           60.0           60.0         60.0         4.815 🔴

💡 Key observations:
   India score: 60.0 | Top score: 100.0 | Gap: 40.00 points
   India ranks last — confirming the need for systemic reform.
```

📊 Step 5 — Visualizations (4 Charts)

```
COUNTRY_COLORS = {
    'Singapore':   '#F59E0B',
    'South Korea': '#F59E0B',
    'China':       '#F59E0B',
    'Japan':       '#F59E0B',
    'Finland':     '#10B981',
    'Canada':      '#10B981',
    'Germany':     '#10B981',
    'UK':          '#10B981',
    'US':          '#10B981',
    'India':       '#EF4444',
}

fig = plt.figure(figsize=(20, 18))
fig.patch.set_facecolor('#F8FAFC')
gs  = gridspec.GridSpec(2, 2, figure=fig, hspace=0.38, wspace=0.32)
fig.suptitle(
    'MODULE 6 — Intelligent Country Ranking System\nGlobal Education System
    fontsize=17, fontweight='bold', color='#1E293B', y=1.01
)
```

```
countries   = base_ranking.index.tolist()
final_scores = base_ranking['Final Score'].values
bar_colors  = [COUNTRY_COLORS.get(c, '#64748B') for c in countries]

# ── Chart 1: Horizontal Ranking Bar ──────────────────────────────────
ax1 = fig.add_subplot(gs[0, 0])
ax1.set_facecolor('#F8FAFC')
bars = ax1.barh(
    countries[::-1], final_scores[::-1],
    color=bar_colors[::-1], edgecolor='white', linewidth=0.6, height=0.68
)
for bar, val, country in zip(bars, final_scores[::-1], countries[::-1]):
    ax1.text(
        bar.get_width() + 0.3,
        bar.get_y() + bar.get_height() / 2,
        f'{val:.2f}',
        va='center', fontsize=10, fontweight='bold', color='#1E293B'
    )
    rank = base_ranking.loc[country, 'Rank']
    ax1.text(
        1, bar.get_y() + bar.get_height() / 2,
        f'#{rank}', va='center', fontsize=9, color='white', fontweight='bold
    )
ax1.set_xlim(0, 115)
ax1.set_xlabel('Final Ranking Score (0-100 scale)', fontsize=11)
ax1.set_title('AI-Generated Country Education Ranking',
              fontsize=12, fontweight='bold', color='#1E293B')
ax1.spines[['top', 'right']].set_visible(False)
ax1.axvline(x=60, color='#CBD5E1', linestyle='--', linewidth=1, alpha=0.8)
patches = [
    mpatches.Patch(color='#10B981', label='Balanced High Performers'),
    mpatches.Patch(color='#F59E0B', label='High-Pressure Performers'),
    mpatches.Patch(color='#EF4444', label='Developing (India)'),
]
ax1.legend(handles=patches, fontsize=8.5, loc='lower right')

# ── Chart 2: Score Breakdown (ML vs Composite) ──────────────────────
ax2 = fig.add_subplot(gs[0, 1])
ax2.set_facecolor('#F8FAFC')
x = np.arange(len(countries))
w = 0.35
ml_scores   = base_ranking['ML Predicted Score'].values
comp_scores = base_ranking['Composite Score'].values
b1 = ax2.bar(x - w/2, ml_scores,   w, label='ML Predicted Score (60%)',
             color='#2563EB', alpha=0.85, edgecolor='white')
b2 = ax2.bar(x + w/2, comp_scores, w, label='Composite Score (40%)',
             color='#10B981', alpha=0.85, edgecolor='white')
ax2.plot(x, final_scores, 'o--', color='#EF4444', linewidth=2,
         markersize=8, label='Final Score (blended)', zorder=5)
ax2.set_xticks(x)
ax2.set_xticklabels(countries, rotation=40, ha='right', fontsize=9)
ax2.set_ylabel('Score', fontsize=11)
ax2.set_title('Score Breakdown: ML vs Composite vs Final',
              fontsize=12, fontweight='bold', color='#1E293B')
ax2.legend(fontsize=8.5)
```

```
ax2.spines[['top', 'right']].set_visible(False)
ax2.set_ylim(50, 110)

# ── Chart 3: Actual G3 vs Predicted Score scatter ──────────────────────
ax3 = fig.add_subplot(gs[1, 0])
ax3.set_facecolor('#F8FAFC')
actual_g3 = base_ranking['Actual G3'].values
scatter_colors = [COUNTRY_COLORS.get(c, '#64748B') for c in countries]
ax3.scatter(actual_g3, final_scores, s=180, c=scatter_colors,
            edgecolor='white', linewidth=1.5, zorder=4)
for i, country in enumerate(countries):
    ax3.annotate(
        country, (actual_g3[i], final_scores[i]),
        textcoords='offset points', xytext=(8, 4),
        fontsize=9, fontweight='bold', color='#1E293B'
    )
z = np.polyfit(actual_g3, final_scores, 1)
x_line = np.linspace(actual_g3.min(), actual_g3.max(), 100)
ax3.plot(x_line, np.poly1d(z)(x_line), '--', color='#64748B',
         linewidth=1.5, alpha=0.7, label='Trend')
ax3.set_xlabel('Actual Average Grade (G3)', fontsize=11)
ax3.set_ylabel('AI Final Ranking Score', fontsize=11)
ax3.set_title('Actual Performance vs AI Ranking Score',
              fontsize=12, fontweight='bold', color='#1E293B')
ax3.legend(fontsize=9)
ax3.spines[['top', 'right']].set_visible(False)

# ── Chart 4: Feature Importance used in ranking ────────────────────────
ax4 = fig.add_subplot(gs[1, 1])
ax4.set_facecolor('#F8FAFC')
fi_sorted = fi_norm.sort_values(ascending=True)
fi_labels = [FEATURE_LABELS[f] for f in fi_sorted.index]
fi_colors = ['#EF4444' if f in NEGATIVE_FEATURES else '#2563EB'
             for f in fi_sorted.index]
bars = ax4.barh(fi_labels, fi_sorted.values,
                color=fi_colors, edgecolor='white', linewidth=0.5, height=0.
for bar, val in zip(bars, fi_sorted.values):
    ax4.text(
        bar.get_width() + 0.003,
        bar.get_y() + bar.get_height() / 2,
        f'{val:.3f}', va='center', fontsize=9, fontweight='bold'
    )
ax4.set_xlabel('Feature Importance Weight', fontsize=11)
ax4.set_title('Feature Weights Used in Ranking Formula',
              fontsize=12, fontweight='bold', color='#1E293B')
ax4.spines[['top', 'right']].set_visible(False)
pos_p = mpatches.Patch(color='#2563EB', label='Positive factor')
neg_p = mpatches.Patch(color='#EF4444', label='Negative factor')
ax4.legend(handles=[pos_p, neg_p], fontsize=9)

plt.savefig('module6_ranking.png', dpi=150, bbox_inches='tight', facecolor='
plt.show()
print('✅ Chart saved as module6_ranking.png')
```



✅ Chart saved as module6_ranking.png

🔄 Step 6 — Dynamic Ranking Simulator

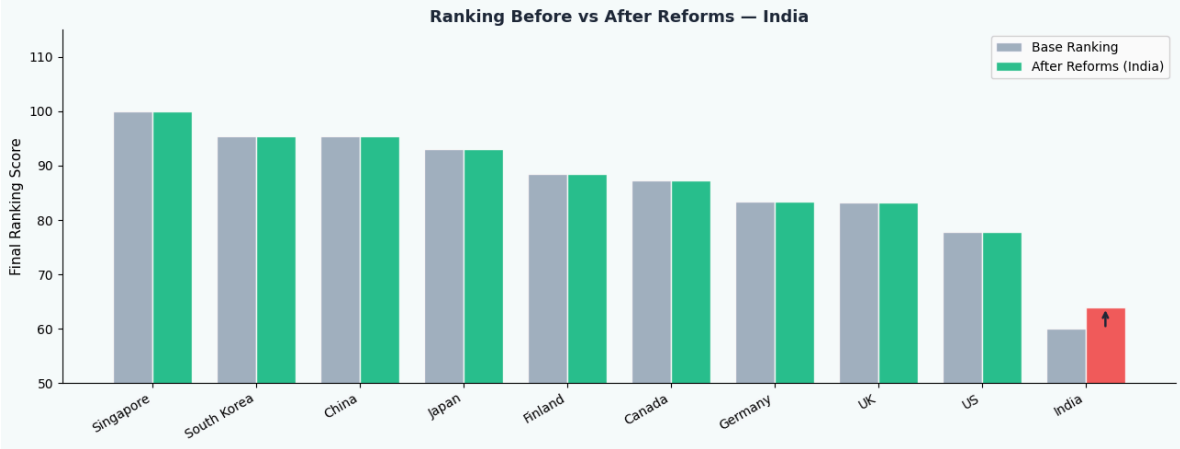
This cell simulates **what happens to India's rank if specific reforms are applied.** The ranking recomputes automatically when you change any variable below.

> Modify the values under `# ── REFORM PARAMETERS` to test different scenarios.

```
# ═══════════════════════════════════════════════════════════════
# REFORM PARAMETERS — change these to test different scenarios
# ═══════════════════════════════════════════════════════════════

REFORMS = {
    # Feature name          : change amount (+positive / -negative)
    'teacher_ratio_index':    +1.5,   # improve teacher quality
    'student_wellbeing_index':+1.0,   # improve student well-being
    'school_quality_index':   +0.5,   # improve school quality
    'failures':               -2.0,   # reduce failures (capped at 0)
    'absences':               -2.0,   # reduce absences (capped at 0)
}

COUNTRY_TO_REFORM = 'India'   # Change this to test any country

# ── Apply reforms ──────────────────────────────────────────────
df_reformed = df.copy()
mask = df_reformed['country'] == COUNTRY_TO_REFORM

for feat, delta in REFORMS.items():
    if feat in df_reformed.columns:
        df_reformed.loc[mask, feat] = (df_reformed.loc[mask, feat] + delta).

# Compute new ranking
new_ranking = compute_ranking(df_reformed)

# ── Print comparison ───────────────────────────────────────────
print(f'Reform applied to: {COUNTRY_TO_REFORM}')
print(f'Reforms: {REFORMS}\n')

print(f'{"":5} {"Country":<15} {"Base Score":>12} {"New Score":>11} {"Change
print('-' * 75)

for country in base_ranking.index:
    base_s = base_ranking.loc[country, 'Final Score']
    new_s  = new_ranking.loc[country, 'Final Score']
    base_r = base_ranking.loc[country, 'Rank']
    new_r  = new_ranking.loc[country, 'Rank']
    delta  = new_s - base_s
    arrow  = '▲' if delta > 0.01 else ('▼' if delta < -0.01 else '=')
    flag   = ' <-- REFORMED' if country == COUNTRY_TO_REFORM else ''
    print(f'  {arrow}  {country:<15} {base_s:>12.2f} {new_s:>11.2f} '
          f'{delta:>+9.2f} {int(base_r):>10} {int(new_r):>9}{flag}')
```

```
print('\n' + '=' * 55)
base_s = base_ranking.loc[COUNTRY_TO_REFORM, 'Final Score']
new_s  = new_ranking.loc[COUNTRY_TO_REFORM, 'Final Score']
base_r = int(base_ranking.loc[COUNTRY_TO_REFORM, 'Rank'])
new_r  = int(new_ranking.loc[COUNTRY_TO_REFORM, 'Rank'])
improvement_pct = ((new_s - base_s) / base_s * 100)
print(f'  {COUNTRY_TO_REFORM} — Score: {base_s:.2f} → {new_s:.2f} '
          f'(+{new_s-base_s:.2f} pts, {improvement_pct:+.1f}%)')
print(f'  {COUNTRY_TO_REFORM} — Rank:  #{base_r} → #{new_r}')
print('=' * 55)

# ── Side-by-side bar chart: Before vs After ──────────────────────────
fig, ax = plt.subplots(figsize=(13, 5))
fig.patch.set_facecolor('#F8FAFC')
ax.set_facecolor('#F8FAFC')

x  = np.arange(len(base_ranking))
w  = 0.38
b_scores = base_ranking['Final Score'].values
n_scores = [new_ranking.loc[c, 'Final Score'] for c in base_ranking.index]

bar_base = ax.bar(x - w/2, b_scores, w, label='Base Ranking',
                  color='#94A3B8', alpha=0.85, edgecolor='white')
bar_new  = ax.bar(x + w/2, n_scores, w, label=f'After Reforms ({COUNTRY_TO_R
                  color=['#EF4444' if c == COUNTRY_TO_REFORM else '#10B981'
                         for c in base_ranking.index],
                  alpha=0.88, edgecolor='white')

for i, (bs, ns) in enumerate(zip(b_scores, n_scores)):
    if abs(ns - bs) > 0.01:
        ax.annotate('', xy=(x[i]+w/2, ns), xytext=(x[i]+w/2, bs),
                    arrowprops=dict(arrowstyle='->', color='#1E293B', lw=1.8

ax.set_xticks(x)
ax.set_xticklabels(base_ranking.index, rotation=30, ha='right', fontsize=10)
ax.set_ylabel('Final Ranking Score', fontsize=11)
ax.set_title(f'Ranking Before vs After Reforms — {COUNTRY_TO_REFORM}',
             fontsize=13, fontweight='bold', color='#1E293B')
ax.set_ylim(50, 115)
ax.legend(fontsize=10)
ax.spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.savefig('module6_dynamic_ranking.png', dpi=150, bbox_inches='tight', fac
plt.show()
print('✅ Chart saved as module6_dynamic_ranking.png')
```

```
Reform applied to: India
Reforms: {'teacher_ratio_index': 1.5, 'student_wellbeing_index': 1.0, 'school

      Country           Base Score   New Score    Change  Base Rank  New Rank
--------------------------------------------------------------------------------
  =  Singapore            100.00      100.00     +0.00          1         1
  =  South Korea           95.39       95.39     +0.00          2         2
  =  China                 95.38       95.38     +0.00          3         3
  =  Japan                 92.99       92.99     +0.00          4         4
  =  Finland               88.53       88.53     +0.00          5         5
  =  Canada                87.23       87.23     +0.00          6         6
  =  Germany               83.41       83.41     +0.00          7         7
  =  UK                    83.28       83.28     +0.00          8         8
  =  US                    77.80       77.80     +0.00          9         9
  ▲  India                 60.00       63.87     +3.87         10        10

=======================================================
  India — Score: 60.00 → 63.87 (+3.87 pts, +6.4%)
  India — Rank:  #10 → #10
=======================================================
```




```
✅ Chart saved as module6_dynamic_ranking.png
```

💾 Step 7 — Download All Results

```
from google.colab import files

base_ranking.to_csv('module6_base_ranking.csv')
new_ranking.to_csv('module6_reformed_ranking.csv')
```

```
fi_norm.to_frame('Weight').to_csv('module6_feature_weights.csv')

print('📥 Downloading files...')
files.download('module6_base_ranking.csv')
files.download('module6_reformed_ranking.csv')
files.download('module6_feature_weights.csv')
files.download('module6_ranking.png')
files.download('module6_dynamic_ranking.png')
print('✅ Done!')
```

```
📥 Downloading files...
✅ Done!
```

✅ Module 6 Complete!

Output Files

File	Contents
`module6_base_ranking.csv`	AI-generated ranking with all scores
`module6_reformed_ranking.csv`	Ranking after applying reforms
`module6_feature_weights.csv`	Feature importance weights used in formula
`module6_ranking.png`	4-panel main visualization
`module6_dynamic_ranking.png`	Before vs After reform comparison chart

Ranking Formula

```
Final Score = 60% x ML Predicted Score  +  40% x Weighted Composite Score
```

Both components normalized to 60-100 using a fixed base scale.

Base Ranking Result

Rank	Country	Score
1	Singapore	100.00
2	South Korea	95.39
3	China	95.38
4	Japan	92.99
5	Finland	88.53
6	Canada	87.23
7	Germany	83.41
8	UK	83.28
9	US	77.80
10	India	60.00